

نظري

6

8

1440 sp

كلية الهندسة المعلوماتية

السنة الثانية

Exception Handling Part 2

د. غاندي هسام



RB Informatics; 13/11/2025

برمجة ٣

في هذه المحاضرة سنتابع بال Exception Handling

- إحدى أهم فوائد ال Exceptions هي أنها تمكّن البرنامج من العمل بشكل طبيعي دون توقف ولا تعطل على الرغم من وجود خطأ او مشكلة فيه وذلك بمجرد كتابة كود يخبر البرنامج كيف يتصرف اذا حدث خطأ فيه .
- كما نعلم أن القسمة على صفر ستؤدي الى حدوث Run Time error أي توقف للبرنامج بشكل كامل ألا أن استخدام ال Exceptions في الكود سيعالج هذه المشكلة .

مثال:

```
// Handle error gracefully and continue.
class ExcDemo3 {
    public static void main(String[] args) {
        int[] number = { 4, 8, 16, 32, 64, 128 };
        int[] denom = { 2, 0, 4, 4, 0, 8 };

        for(int i = 0; i < number.length; i++) {
            try {
                System.out.println(number[i] + " / " +
                    denom[i] + " is " +
                    number[i] / denom[i]);
            }

            catch (ArithmeticException exc) {
                // catch the exception
                System.out.println("Can't divide by Zero!");
            }
        }
    }
}
```

Output:

```
4 / 2 is 2
Can't divide by Zero!
16 / 4 is 4
32 / 4 is 8
Can't divide by Zero!
128 / 8 is 16
```

في الكود السابق كان لدينا مصفوفتان وتريد قسمة كل عنصر في الأولى على العنصر المناظر له في الثانية، وبما أنه من الممكن أن يحتوي أحد عناصر المصفوفة الثانية على صفر قمنا باستخدام ال Exceptions حيث يقوم بتوليد Exceptions من النوع ArithmeticException لأنه خطأ رياضي ، وأثناء التنفيذ يتم التعامل مع هذا الاستثناء عن طريق الإبلاغ عن الخطأ و طباعة رسالة للمستخدم تفيد بأنه حدث خطأ (مثل: "لا يمكن القسمة على الصفر") ثم متابعة التنفيذ بلا توقف .

Multiple catch Statements

- هل يمكنك وضع أكثر من عبارة catch في الكود ؟
- نعم يمكنك ربط أكثر من عبارة catch مع كتلة try واحدة ولكن بشرط أن تتعامل كل عبارة catch مع نوع مختلف و مخصص من الاستثناءات أي أنه لا يمكنك تكرار نفس الخطأ في أكثر catch واحدة .

مثال

```
// Use multiple catch statements.
class ExcDemo4 {
    public static void main(String[] args) {
        // Here, number is longer than denominator.
        int[] number = { 4, 8, 16, 32, 64, 128, 256, 512 };
        int[] denom = { 2, 0, 4, 4, 0, 8 };

        for(int i = 0; i < number.length; i++) {
            try {
                System.out.println(number[i] + " / " +
                    denom[i] + " is " +
                    number[i] / denom[i]);
            }
            catch (ArithmeticException exc) {
                // catch the exception
                System.out.println("Can't divide by Zero!");
            }
            catch (ArrayIndexOutOfBoundsException exc) {
                // catch the exception
                System.out.println("No matching element found.");
            }
        }
    }
}
```

Output:

```
4 / 2 is 2
Can't divide by Zero!
16 / 4 is 4
32 / 4 is 8
Can't divide by Zero!
128 / 8 is 16
No matching element found.
No matching element found.
```



■ في المثال السابق:

- البرنامج يعالج أخطاء تجاوز حدود المصفوفة (ArrayIndexOutOfBoundsException)، بالإضافة إلى أخطاء القسمة على الصفر (ArithmeticException)
- أي أن نفس كتلة try قد تحتوي على أكثر من عملية قد ينتج عنها أكثر من نوع من الأخطاء، لذا نضع catch لكل نوع خطأ، وبذلك يستطيع البرنامج معرفة أي خطأ حدث والتعامل معه بشكل مناسب دون التوقف.

- الفئة الأب (superclass) لجميع الاستثناءات في Java هي Throwable، فإذا أردت التقاط جميع الاستثناءات الممكنة والمحتملة من ال (subclasses)، يمكنك استخدام catch مع Throwable وهذا ما يعد شاملاً.

ملاحظة

- استخدام (Throwable exc) catch يعد شامل برمجياً ولكن من الناحية الهندسية فهو خاطئ
- الكلاس Throwable يلتقط أيضاً ال Errors مثل OutOfMemoryError هو خطأ كارثي من السيء ان البرنامج يعالج هذا الخطأ ويكمل عمله

ملاحظة:

إذا كان هناك حاجة للتقاط استثناءات شاملة من Throwable و مخصصة من ال classes الفرعية منها يجب عليك أن تراعي وجود عبارة ال catch الخاصة بال classes الفرعية أولاً ، أي (يجب التعامل مع الأخطاء الأكثر تحديداً أولاً، ثم التعامل مع الأخطاء العامة بعدها)

ما السبب؟

- وضع catch الخاصة بالاب أولاً سيؤدي إلى وجود كود غير قابل للوصول (unreachable code) لأن catch الخاصة بال classes الفرعية لن يتم تنفيذها أبداً، وفي Java وجود كود غير قابل للوصول يُعتبر خطأ.

```
// Subclasses must precede superclasses in catch statements.
class ExcDemo5 {
    public static void main(String[] args) {
        // Here, numer is longer than denom.
        int[] numer = { 4, 8, 16, 32, 64, 128, 256, 512 };
        int[] denom = { 2, 0, 4, 4, 0, 8 };
        for(int i = 0; i < numer.length; i++) {
            try {
                System.out.println(numer[i] + " / " + denom[i] + " is " + numer[i] / denom[i]);
            }
            catch (ArrayIndexOutOfBoundsException exc) { // catch the exception
                System.out.println("No matching element found.");
            }
            catch (Throwable exc) {
                System.out.println("Some exception occurred.");
            }
        }
    }
}
```

Output:

```
4 / 2 is 2
```

```
Some exception occurred.
```

```
16 / 4 is 4
```

```
32 / 4 is 8
```

```
Some exception occurred.
```

```
128 / 8 is 16
```

```
No matching element found.
```

```
No matching element found.
```



رمي الاستثناء (Throwing an Exception)

- في الأمثلة السابقة، كانت الاستثناءات التي تمت معالجتها يتم توليدها تلقائياً بواسطة الآلة الافتراضية لجافا (JVM).
- ولكن، من الممكن رمي استثناء بشكل يدوي باستخدام التعليمة (throw) ويكون استخدامها على الشكل :
`throw exceptOb;`
- (حيث يجب أن يكون الكائن `exceptOb` كائناً من إحدى فئات الاستثناءات (Exception Object) المشتقة من الفئة الأساسية (Throwable) .

فائدة اربيسيزية ☺ :

في بعض الأحيان يحتاج المبرمج إلى إنشاء استثناء بنفسه ليعبر عن خطأ معين يحدث خلال تنفيذ البرنامج. هنا يأتي دور التعليمة `throw`

ما معنى `throw` ؟

هي كلمة مفتاحية تسمح لك برمي استثناء بشكل يدوي داخل الكود. عند تنفيذها، يتوقف مسار البرنامج الطبيعي وينتقل مباشرة إلى أقرب كتلة `catch` قادرة على معالجة هذا الاستثناء.

```
// Manually throw an exception.
class ThrowDemo {
    public static void main(String[] args) {
        try {
            System.out.println("Before throw.");
            throw new ArithmeticException();
        }
        catch (ArithmeticException exc) {
            // catch the exception
            System.out.println("Exception
caught.");
        }
        System.out.println("After try/catch block.");
    }
}
```

Output:

Before throw.

Exception caught.

After try/catch block.

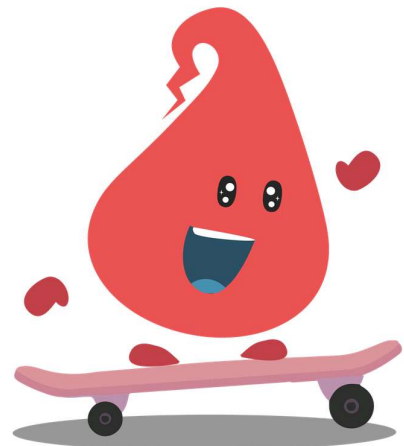
في المثال السابق نلاحظ عدم حدوث خطأ فعلي في الكود من النوع ArithmeticException ولكن كتلة ال try تقوم برمي خطأ من هذا النوع بإرادة المبرمج وذلك باستخدام التعليمة

```
throw new ArithmeticException();
```

:Using finally

■ كتلة finally في جافا تُستخدم لضمان تنفيذ جزء معين من الكود بعد تنفيذ كتلة try و ال catch

```
try {
    // block of code to monitor for errors
}
catch (ExceptionType1 exOb) {
    // handler for ExceptionType1
}
catch (ExceptionType2 exOb) {
    // handler for ExceptionType2
}
// ...
finally {
    // finally code
}
```



فائدة ارييسيزية 😊 :

ال finally ضرورية لأنه عندما يقوم برنامجك بفتح ملف أو استخدام أي مورد خارجي، فمن الضروري إغلاق هذا المورد بعد الانتهاء منه حتى لو حدث خطأ ، و بدون finally والتعليمات داخلها، قد تبقى هذه الموارد مفتوحة .

Using throws

- في بعض الحالات ممكن أن تكتب دالة وهذه الدالة تقوم بتوليد Exception لكن لا تريد لهذه الدالة أن تقوم بمعالجته داخلها (أي رميه والتقاطه) وبالتالي يجب عليك أن تصرّح عن ذلك الاستثناء باستخدام عبارة throws

الصيغة العامة لذلك:

```
return-type method-name(param-list) throws except-list {
    // body of method
}
```

except-list هنا تعني أنه يمكن لهذه الدالة أ، ترمي العديد من الاستثناءات ونكتبها جميعها بعد كلمة throws حيث يتم الفصل بينها بواسطة الفاصلة (,).

■ هناك نوعين من الاستثناءات :

1. Unchecked Exceptions :

- هي الاستثناءات التي:
- هي حصراً الكلاسات التي ترث من RuntimeException
- لا يتحقق المترجم من وجود معالجة لها (بواسطة try\catch) وفي حال عدم وجود معالجة لها سيتوقف البرنامج عن التنفيذ (Runtime Error) او وجود تصريح عنها بواسطة throws .
- منها جميع الاستثناءات السابقة التي تعملنا معها :
 - ArithmeticException
 - NullPointerException
 - ArrayIndexOutOfBoundsException

2. Checked Exceptions :

- هي الاستثناءات التي:
- يجب التصريح عنها باستخدام throws إذا لم تتم معالجتها داخل الدالة.
- تجاهلها يؤدي إلى خطأ ترجمة (compile-time error) .

```
// Use throws.
class ThrowsDemo {
    public static char prompt(String str)
        throws java.io.IOException { // Notice the throws
        System.out.print(str + ": ");
        return (char) System.in.read();
    }

    public static void main(String[] args) {
        char ch;
        try {
            ch = prompt("Enter a letter");
        }
        catch(java.io.IOException exc) {
            System.out.println("I/O exception occurred.");
            ch = 'X';
        }
        // Since prompt() might throw an exception, a call to it must
        be enclosed within a try
        System.out.println("You pressed " + ch);
    }
}
```

الكود السابق يوضح كيفية رمي Checked Exception (IOException) باستخدام throws، وكيفية معالجة الاستثناء باستخدام try/catch لضمان استمرار تنفيذ البرنامج بأمان

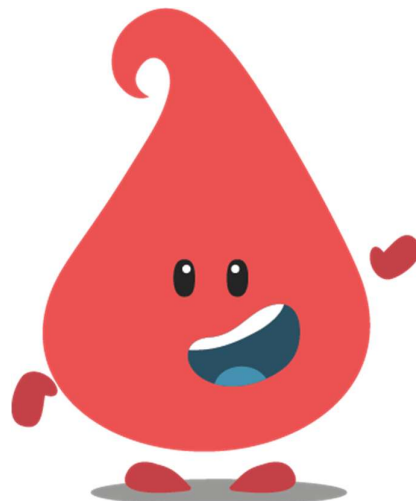
▪ فائدة اربيسيزية 😊 :

▪ IOException هو استثناء متعلق بعمليات الإدخال/الإخراج.

Multi-catch

- كما تعلمنا سابقاً، من الممكن أن يتبع بلوك try أكثر من catch
- في بعض الحالات، تقوم أكثر من catch بتنفيذ نفس سلسلة الأوامر على الرغم من أنها تلتقط أنواع استثناءات مختلفة لذلك يمكن استخدام catch واحد للتعامل مع هذه الاستثناءات دون تكرار الكود و يتم ذلك عن طريق فصل كل نوع استثناء في القائمة باستخدام عامل (|) OR

```
class MultiCatch {  
    public static void main(String[] args) {  
        int a = 88, b = 0;  
        int result;  
        char[] chrs = { 'A', 'B', 'C' };  
        for (int i = 0; i < 2; i++) {  
            try {  
                if (i == 0)  
                    result = a / b; // generate an ArithmeticException  
                else  
                    chrs[5] = 'X'; // generate an ArrayIndexOutOfBoundsException  
                // This catch clause catches both exceptions.  
            }  
            catch (ArithmeticException | ArrayIndexOutOfBoundsException e)  
            {  
                System.out.println("Exception caught: " + e);  
            }  
        }  
        System.out.println("After multi-catch.");  
    }  
}
```



See You Later